RETAILMART V3 · SQL

Indexing for Analysts -- Hands-On

— Siraj Sir

📱 WhatsApp Announcement

Indexing for Analysts (Hands-On)

Yesterday you learned to READ query plans. Today you learn to FIX the slow ones. Indexes are the number one tool for making queries fast. You will create indexes, measure the before/after, and understand when they help vs when they do not.

Part 1: B-Tree Indexes -- The Default and Most Common
Part 2: Composite and Partial Indexes
Part 3: When Indexing Helps vs When It Does Not

This is hands-on: you will CREATE INDEX, run EXPLAIN ANALYZE before and after, and see the speed difference yourself.

Prerequisites: Query Plans (EXPLAIN, EXPLAIN ANALYZE, scan types). Today you fix what yesterday diagnosed.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: B-Tree Indexes	2 Hours	What an index is and how B-Tree works, CREATE INDEX syntax, Before/after EXPLAIN ANALYZE comparison
Part 2: Composite & Partial Indexes		Multi-column indexes and leftmost prefix rule, Partial indexes with WHERE clause, Choosing the right index for your query
Part 3: When Indexing Helps vs Not		Full-table aggregates ignore indexes, Index overhead on writes, Finding existing indexes with pg_indexes

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Setup & RetailMart
- ✓ DDL & Data Types
- ✓ Constraints & DML
- ✓ Filtering & Sorting
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregation
- ✓ Self Join & UNION
- ✓ Subqueries Part 1
- ✓ Subqueries & CTEs
- ✓ Database Concepts
- ✓ Review & Practice
- ✓ Window Funcs Part 1
- ✓ Aggregation & Frames
- ✓ LAG & LEAD

RECAP

What We Covered Yesterday

- EXPLAIN shows the plan; EXPLAIN ANALYZE shows actual execution
- Scan types: Seq Scan, Index Scan, Bitmap Scan
- Join algorithms: Nested Loop, Hash Join, Merge Join
- Sargability: keep WHERE columns bare for index usage

STORY

The Library Card Catalog

Imagine a library with 50,000 books. Without a catalog, finding one book means checking every shelf. With a card catalog sorted by title, you flip to 'M', then 'Mo', then 'Moby Dick' -- three lookups instead of 50,000. A B-Tree index works exactly the same way for your database columns.

DEFINITION

Database Index

- A separate data structure that maps column values to row locations
- Speeds up reads (SELECT, WHERE, JOIN, ORDER BY)
- Slows down writes slightly (INSERT, UPDATE, DELETE must update the index too)
- Takes extra disk space (typically 10-30% of table size)

B-Tree -- The Default Index Type

B-Tree (Balanced Tree) is PostgreSQL's default index. It supports:

- Equality: WHERE col = value
- Range: WHERE col > value, WHERE col BETWEEN a AND b
- Sorting: ORDER BY col (avoids explicit sort)
- Prefix matching: WHERE col LIKE 'abc%'

B-Tree does NOT support:

- Leading wildcard: WHERE col LIKE '%abc'
- Functions on columns: WHERE UPPER(col) = 'X' (unless expression index)

Index Commands

```
-- Basic B-Tree

CREATE INDEX idx_name
  ON schema.table (column);

-- Composite (multi-column)

CREATE INDEX idx_name
  ON schema.table (col1, col2);

-- Partial (conditional)

CREATE INDEX idx_name
  ON schema.table (column)
  WHERE condition;
```

Index Commands

```
-- Concurrently (no lock)
CREATE INDEX CONCURRENTLY idx_name
  ON schema.table (column);
  -- Does not block writes
```

Hands-On: Before and After

Step 1: Check the current plan (before index):

Before Index

```
EXPLAIN ANALYZE
SELECT order_id, order_date, net_total
FROM sales.orders
WHERE cust_id = 42;

-- Expected: Seq Scan (no index on cust_id)
-- Time: ~80ms (scans all 150,000 rows)
```

Step 2: Create the index:

Create Index

```
CREATE INDEX idx_orders_cust_id
  ON sales.orders (cust_id);
```

Step 3: Check the plan again (after index):

After Index

```
EXPLAIN ANALYZE
SELECT order_id, order_date, net_total
FROM sales.orders
WHERE cust_id = 42;

-- Expected: Index Scan using idx_orders_cust_id
-- Time: ~0.5ms (jumps directly to matching rows)
-- Speedup: ~160x faster!
```

PRO TIP

PRO TIP

Always name your indexes descriptively: idx_tablename_column1_column2. This makes it easy to identify them later in pg_indexes and DROP INDEX commands.

Practice 1: Single Column Index

EASY

SCENARIO: Analysts frequently filter orders by status, but the query is slow.

TASK: (1) Run EXPLAIN ANALYZE on the query below and note the scan type and execution time. (2) Create a B-Tree index on the status column. (3) Re-run EXPLAIN ANALYZE and compare.

HINT: CREATE INDEX idx_orders_status ON sales.orders (order_status);

Query

✅ ANSWER

```
EXPLAIN ANALYZE
SELECT order_id, order_date, order_status
FROM sales.orders
WHERE order_status = 'Cancelled';
```

Answer

✅ ANSWER

```
-- Step 1: Before
-- Seq Scan on orders  (actual time=0.01..85ms rows=~7500)

-- Step 2: Create index
CREATE INDEX idx_orders_status
  ON sales.orders (order_status);

-- Step 3: After
-- Bitmap Index Scan on idx_orders_status
-- (actual time=0.5..15ms rows=~7500)

-- Note: PostgreSQL may choose Bitmap Scan instead of
-- Index Scan because ~5% of rows match 'Cancelled'.
-- Bitmap is better than Index Scan for medium selectivity.
```

Practice 2: Index for JOIN Performance

MEDIUM

SCENARIO: A multi-table join is slow because the join column lacks an index.

TASK: (1) Run EXPLAIN ANALYZE on the join below. (2) Check if order_items.order_id has an index. (3) If not, create one and measure the improvement.

HINT: Check for existing indexes first: SELECT indexname FROM pg_indexes WHERE tablename = 'order_items';

Query

✅ ANSWER

```
EXPLAIN ANALYZE
SELECT o.order_id, o.order_date,
       SUM(oi.quantity * oi.unit_price) AS total
FROM sales.orders o
JOIN sales.order_items oi
  ON o.order_id = oi.order_id
WHERE o.order_id BETWEEN 1000 AND 1100
GROUP BY o.order_id, o.order_date;
```

Answer

✅ ANSWER

```
-- Check existing indexes:
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'order_items';

-- If no index on order_id:
CREATE INDEX idx_order_items_order_id
  ON sales.order_items (order_id);

-- Before: Hash Join with Seq Scan on order_items
-- After: Nested Loop with Index Scan on order_items
-- The small range (100 orders) + index = Nested Loop
```

STORY

The Phone Book Analogy

A phone book is sorted by last name, then first name. If you search for 'Sharma, Priya' -- fast. If you search for just 'Priya' (first name only) -- you must scan the entire book. This is the leftmost prefix rule: a composite index on (last_name, first_name) helps queries on last_name alone, but NOT queries on first_name alone.

Composite Indexes

A composite index covers multiple columns. Column ORDER matters:

Composite Index

```
-- Index on (store_id, order_date)
CREATE INDEX idx_orders_store_date
  ON sales.orders (store_id, order_date);

-- This index helps:
-- WHERE store_id = 5                    (leftmost column)
-- WHERE store_id = 5 AND order_date > X (both columns)
-- WHERE store_id = 5 ORDER BY order_date (filter + sort)

-- This index does NOT help:
-- WHERE order_date > X                  (skips leftmost!)
-- ORDER BY order_date                   (skips leftmost!)
```

DEFINITION

Leftmost Prefix Rule

- A composite index on (A, B, C) supports queries on: A, (A,B), or (A,B,C)
- It does NOT support queries on: B alone, C alone, or (B,C)
- Think of it like a phone book: you must know the last name to use it efficiently

Choosing Column Order

Put the most selective (most filtered) column first:

Column Order Example

```
-- Query: WHERE order_status = 'Cancelled' AND store_id = 5

-- Option A: (status, store_id)
-- 'Cancelled' = 5% of rows -> 7,500 rows
-- Then filter store_id within those -> ~38 rows

-- Option B: (store_id, status)
-- store_id = 5 -> 750 rows (1 of 200 stores)
-- Then filter status within those -> ~38 rows

-- Both reach ~38 rows, but Option B starts smaller.
-- General rule: put the column with fewer matching rows first.
-- Exception: if you also ORDER BY one column, put it second.
```

Partial Indexes

A partial index only indexes rows that match a WHERE condition:

Partial Index

```
-- Only index active (non-delivered) orders
CREATE INDEX idx_orders_pending
  ON sales.orders (order_date)
  WHERE order_status IN ('Processing', 'Shipped', 'Out for Delivery');

-- This index is SMALL (only ~30% of rows)
-- and is used ONLY for queries that match the condition:

-- Uses the partial index:
SELECT * FROM sales.orders
WHERE order_status = 'Processing'
  AND order_date > '2025-06-01';

-- Does NOT use the partial index:
SELECT * FROM sales.orders
WHERE order_status = 'Delivered'
  AND order_date > '2025-06-01';
```

PRO TIP

PRO TIP

Partial indexes are perfect for 'active records' patterns. If 90% of orders are delivered and you always query the other 10%, a partial index on the non-delivered rows is small and fast.

Practice 3: Composite Column Order

MEDIUM

SCENARIO: Analysts frequently run queries filtering by both region and order_date.

TASK: Create a composite index to speed up this query. Choose the right column order and explain your reasoning. Verify with EXPLAIN ANALYZE before and after.

HINT: The region filter is on the stores table, not orders. Think about which table needs the index. The join condition and filter together determine the best strategy.

Query

✅ ANSWER

```
EXPLAIN ANALYZE
SELECT o.order_id, o.order_date, o.net_total
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE dr.region_name = 'West'
  AND o.order_date >= '2025-06-01';
```

Answer

✅ ANSWER

```
-- The region filter is on stores, not orders.
-- An index on stores.region would help, but stores
-- is small (200 rows) -- probably already fast.

-- The bigger win: index on orders(store_id, order_date)
CREATE INDEX idx_orders_store_date
  ON sales.orders (store_id, order_date);

-- This helps the join (store_id) and the date filter
-- in a single index lookup.
-- store_id first because it's the join key.
-- order_date second to further narrow within each store.
```

Practice 4: Partial Index for Active Records

HARD

SCENARIO: The support team constantly queries open tickets but never searches resolved ones. The tickets table has 20,000 rows, 85% are resolved.

TASK: Create a partial index that only covers open tickets. Compare the index size to a full index. Verify with EXPLAIN ANALYZE.

HINT: CREATE INDEX ... ON support.tickets (created_date) WHERE status IN ('Open', 'In Progress');

Query

✅ ANSWER

```
EXPLAIN ANALYZE
SELECT ticket_id, customer_id, created_date
FROM support.tickets
WHERE status IN ('Open', 'In Progress')
ORDER BY created_date DESC;
```

Answer

✅ ANSWER

```
-- Create partial index:
CREATE INDEX idx_tickets_open
  ON support.tickets (created_date DESC)
  WHERE status IN ('Open', 'In Progress');

-- Size comparison:
-- Full index on (created_at): indexes all 20,000 rows
-- Partial index: indexes only ~3,000 rows (15%)
-- About 7x smaller!

-- EXPLAIN ANALYZE should show:
-- Index Scan Backward using idx_tickets_open
-- The DESC in the index matches ORDER BY ... DESC.
```

When Indexes DO Help

- WHERE col = value on a large table with low selectivity (few matching rows)
- JOIN conditions between large tables
- ORDER BY col when returning a small subset (with LIMIT)
- DISTINCT on a column (index provides pre-sorted data)

When Indexes Do NOT Help

- Full-table aggregates: SELECT COUNT(*) FROM big_table -- must read every row anyway
- Queries returning most of the table: WHERE order_status = 'Delivered' on 55% of rows
- Small tables (< 1000 rows): Seq Scan is already fast
- Columns with very low cardinality: boolean columns with 50/50 split

Index Does Not Help Example

```
-- Full aggregate: index is useless
EXPLAIN ANALYZE
SELECT COUNT(*), AVG(net_total)
FROM sales.orders;
-- Seq Scan (must read every row for COUNT and AVG)
-- An index on net_total would not help here.

-- Low selectivity: index may be SLOWER
EXPLAIN ANALYZE
SELECT * FROM sales.orders
WHERE order_status = 'Delivered';
-- 55% of rows match -> Seq Scan is faster
-- than bouncing between index and table 82,000 times.
```

Check Existing Indexes

```
-- List all indexes on a table:
SELECT indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'sales'
  AND tablename = 'orders'
ORDER BY indexname;

-- Index size:
SELECT pg_size_pretty(pg_relation_size('orders_pkey'))
  AS index_size;

-- Table size vs total (with indexes):
SELECT
  pg_size_pretty(pg_relation_size('sales.orders'))
    AS table_size,
  pg_size_pretty(pg_total_relation_size('sales.orders'))
    AS total_with_indexes;
```

Redundant Indexes

An index on (A) is redundant if you already have an index on (A, B) -- the composite index covers single-column lookups on A.

Redundant Index Example

```
-- If you have:
CREATE INDEX idx_a_b ON orders (store_id, order_date);

-- Then this is REDUNDANT:
CREATE INDEX idx_a ON orders (store_id);
-- The composite index already covers store_id-only queries.

-- But this is NOT redundant:
CREATE INDEX idx_b ON orders (order_date);
-- The composite index does NOT help order_date-only queries
-- (leftmost prefix rule).
```

COMMON MISTAKE

Common Indexing Mistakes

Indexing everything: Each index slows down INSERT/UPDATE/DELETE. Index only the columns your queries actually filter, join, or sort on.

Wrong column order in composite index: Remember the leftmost prefix rule. Put the most filtered or joined column first.

Indexing low-cardinality columns: A boolean column with 50/50 distribution gains nothing from a B-Tree index.

Forgetting about expression indexes: If you must use WHERE UPPER(col) = X, create an expression index: CREATE INDEX … ON table (UPPER(col));

Practice 5: Identify the Redundant Index

HARD

SCENARIO: A junior DBA created multiple indexes on the orders table. Some may be redundant.

TASK: Look at the four indexes below. Identify which ones are redundant and can be safely dropped. Explain why.

HINT: Check if any single-column index is a prefix of a multi-column index. The multi-column index covers the single-column queries.

Existing Indexes

✅ ANSWER

```
-- Index 1:
CREATE INDEX idx_a ON sales.orders (store_id);

-- Index 2:
CREATE INDEX idx_b ON sales.orders (store_id, order_date);

-- Index 3:
CREATE INDEX idx_c ON sales.orders (cust_id);

-- Index 4:
CREATE INDEX idx_d ON sales.orders (cust_id, order_status);
```

Answer

✅ ANSWER

```
-- REDUNDANT: idx_a (store_id)
-- Reason: idx_b (store_id, order_date) covers
-- all store_id-only queries via leftmost prefix.
-- DROP INDEX idx_a;

-- REDUNDANT: idx_c (customer_id)
-- Reason: idx_d (customer_id, status) covers
-- all customer_id-only queries via leftmost prefix.
-- DROP INDEX idx_c;

-- KEEP: idx_b and idx_d
-- They are not subsets of each other.
```

Practice 6: Full Lab -- Diagnose and Fix a Slow Query

CRAZY

SCENARIO: An analyst reports that their daily revenue-by-category report takes 8 seconds. Your job: diagnose with EXPLAIN ANALYZE, create the right index, and measure the improvement.

TASK: (1) Run EXPLAIN ANALYZE on the query. (2) Identify the bottleneck node. (3) Create one index to fix it. (4) Re-run EXPLAIN ANALYZE and report the speedup. (5) Check if your index is redundant with any existing indexes.

HINT: The two filters are status and order_date. A composite index on (status, order_date) would let PostgreSQL use an Index Scan instead of a Seq Scan on orders.

Slow Query

✅ ANSWER

```
SELECT
  cat.category_name,
  DATE_TRUNC('month', o.order_date) AS month,
  SUM(oi.quantity * oi.unit_price) AS revenue
FROM sales.order_items oi
JOIN sales.orders o ON oi.order_id = o.order_id
JOIN products.products p ON oi.prod_id = p.product_id
JOIN core.dim_brand b ON p.brand_id = b.brand_id
JOIN core.dim_category cat
  ON b.category_id = cat.category_id
WHERE o.order_status = 'Delivered'
  AND o.order_date >= '2025-01-01'
GROUP BY cat.category_name,
  DATE_TRUNC('month', o.order_date)
ORDER BY month, revenue DESC;
```

Answer

✅ ANSWER

```
-- Step 1: EXPLAIN ANALYZE (before)
-- Likely shows Seq Scan on orders with filter

-- Step 2: Bottleneck is Seq Scan on orders
-- (150K rows scanned, only ~40K match the filter)

-- Step 3: Create composite index
CREATE INDEX idx_orders_status_date
  ON sales.orders (order_status, order_date);

-- Step 4: EXPLAIN ANALYZE (after)
-- Should show: Index Scan using idx_orders_status_date
-- or Bitmap Index Scan on idx_orders_status_date
-- Expected speedup: 3-5x

-- Step 5: Check for redundancy
SELECT indexname, indexdef FROM pg_indexes
WHERE tablename = 'orders';
-- If idx_orders_status exists, it is now redundant
-- (covered by idx_orders_status_date).
```

INTERVIEW

Q: What is a B-Tree index and when would you use one?

A: B-Tree (Balanced Tree) is PostgreSQL's default index type. It stores column values in a sorted tree structure, enabling O(log N) lookups. Use it for equality (=), range (<, >, BETWEEN), sorting (ORDER BY), and prefix matching (LIKE 'abc%'). It does not help with leading wildcard searches or function-wrapped columns.

INTERVIEW

Q: What is the leftmost prefix rule?

A: A composite index on (A, B, C) can satisfy queries on A alone, (A, B), or (A, B, C), but NOT on B alone or C alone. The index is sorted by A first, then B within each A value, then C within each (A, B) pair. Skipping the leftmost column means the index cannot be traversed efficiently.

INTERVIEW

Q: When does adding an index NOT help performance?

A: Indexes do not help when: (1) the query reads most of the table (Seq Scan is faster), (2) full-table aggregates like COUNT(*) or AVG() must touch every row, (3) the table is small (< 1000 rows), or (4) the indexed column has very low cardinality (like a boolean). Indexes also add overhead to writes.

HW 1: Create and Verify a B-Tree Index

EASY

SCENARIO: Customer lookups by email are slow.

TASK: Create a B-Tree index on customers.customers(email). Run EXPLAIN ANALYZE before and after on: SELECT * FROM customers.customers WHERE email = 'test@example.com'; Report the scan type change and time difference.

HINT: Before: Seq Scan. After: Index Scan. Note the execution time in both cases.

Answer

✅ ANSWER

```
-- Before: EXPLAIN ANALYZE shows Seq Scan

CREATE INDEX idx_customers_email
  ON customers.customers (email);

-- After: EXPLAIN ANALYZE shows Index Scan
-- using idx_customers_email
-- Speedup: from ~30ms to ~0.1ms for single row lookup
```

HW 2: Composite Index Column Order

MEDIUM

SCENARIO: A query filters by brand and sorts by price.

TASK: Create a composite index to optimize this query. Explain why you chose the column order you did.

HINT: The WHERE filter should be the first column. The ORDER BY column should be second (with DESC direction).

Query

✅ ANSWER

```
SELECT product_name, brand_id, price
FROM products.products
WHERE brand_id = 5
ORDER BY price DESC
LIMIT 10;
```

Answer

✅ ANSWER

```
CREATE INDEX idx_products_brand_price
  ON products.products (brand_id, price DESC);

-- brand_id first: satisfies the WHERE filter
-- price DESC second: satisfies the ORDER BY
-- With LIMIT 10, PostgreSQL can read the first 10 entries
-- from the index without sorting at all!

-- Plan should show:
-- Index Scan Backward using idx_products_cat_price
-- No Sort node needed!
```

HW 3: Partial Index Creation

MEDIUM

SCENARIO: The finance team frequently queries only high-value returns (large refunds).

TASK: Create a partial index on sales.returns that only indexes high-value returns (refund_amount > 5000). Then verify it is used for the team's query.

HINT: The WHERE clause in the index must match the query's filter.

Query

✅ ANSWER

```
SELECT return_id, order_id, return_date, refund_amount
FROM sales.returns
WHERE refund_amount > 5000
ORDER BY return_date DESC;
```

Answer

✅ ANSWER

```
CREATE INDEX idx_returns_highvalue
  ON sales.returns (return_date DESC)
  WHERE refund_amount > 5000;

-- Verify:
EXPLAIN ANALYZE
SELECT return_id, order_id, return_date, refund_amount
FROM sales.returns
WHERE refund_amount > 5000
ORDER BY return_date DESC;

-- Should show: Index Scan using idx_returns_pending
-- Much smaller than a full index on (return_date).
```

HW 4: Diagnose an Unused Index

HARD

SCENARIO: You created an index but EXPLAIN shows PostgreSQL ignores it.

TASK: Create this index, then run the query. Explain why PostgreSQL might choose NOT to use the index.

HINT: 55% of orders are 'Delivered'. When would a Seq Scan be faster than an Index Scan?

Setup

✅ ANSWER

```
CREATE INDEX idx_orders_status2
  ON sales.orders (order_status);

EXPLAIN ANALYZE
SELECT * FROM sales.orders
WHERE order_status = 'Delivered';
```

Answer

✅ ANSWER

```
-- PostgreSQL chooses Seq Scan even with the index!
-- Reason: 'Delivered' matches ~55% of rows (82,500).

-- Index Scan reads: index page -> table page -> repeat
-- For 82,500 rows, that's 82,500 random I/O operations.

-- Seq Scan reads: table pages sequentially (one pass).
-- Sequential I/O is much faster than random I/O.

-- Rule: if more than ~10-15% of rows match,
-- PostgreSQL often prefers Seq Scan.

-- The same index IS useful for:
-- WHERE order_status = 'Cancelled' (~5% of rows)
```

HW 5: Inventory Index Audit

HARD

SCENARIO: The inventory team's queries are slow. Audit the existing indexes on products.inventory and suggest improvements.

TASK: (1) List all existing indexes on products.inventory. (2) Run EXPLAIN ANALYZE on the query below. (3) Suggest one new index to improve it.

HINT: List indexes with SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'inventory';

Query

✅ ANSWER

```
SELECT i.product_id, p.product_name,
       i.store_id, i.quantity_on_hand
FROM products.inventory i
JOIN products.products p ON i.product_id = p.product_id
WHERE i.store_id = 10
  AND i.quantity_on_hand < 5
ORDER BY i.quantity_on_hand;
```

Answer

✅ ANSWER

```
-- Step 1: List indexes
SELECT indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'products'
  AND tablename = 'inventory';

-- Step 2: EXPLAIN ANALYZE the query
-- Likely shows Seq Scan on inventory

-- Step 3: Create composite index
CREATE INDEX idx_inventory_store_qty
  ON products.inventory (store_id, quantity_on_hand);

-- store_id first (equality filter),
-- quantity second (range filter + ORDER BY).
-- This covers both WHERE conditions and the sort.
```

HW 6: Index Strategy Report

CRAZY

SCENARIO: You are asked to write a brief index strategy for the sales.orders table. Review the workload and recommend which indexes to keep, add, or remove.

TASK: Given these common query patterns, recommend the minimal set of indexes: (1) WHERE customer_id = X, (2) WHERE store_id = X AND order_date > Y, (3) WHERE order_status = 'Cancelled', (4) ORDER BY order_date DESC LIMIT N, (5) WHERE customer_id = X AND order_status = 'Delivered'. List each index with its column order and explain why.

HINT: Look for indexes that serve multiple query patterns. Use composite indexes to cover both WHERE and ORDER BY needs.

Answer

✅ ANSWER

```
-- Recommended minimal index set:

-- 1. idx_orders_customer_status
--    ON sales.orders (customer_id, status)
--    Serves: queries 1 and 5

-- 2. idx_orders_store_date
--    ON sales.orders (store_id, order_date)
--    Serves: query 2

-- 3. idx_orders_status (partial: non-Delivered only)
--    ON sales.orders (status)
--    WHERE order_status != 'Delivered'
--    Serves: query 3 (Cancelled is ~5%, good selectivity)

-- 4. idx_orders_date_desc
--    ON sales.orders (order_date DESC)
--    Serves: query 4

-- Total: 4 indexes covering 5 query patterns.
-- No single-column indexes that are redundant
-- with the composites.
```

KEY TAKEAWAYS

KEY TAKEAWAYS

B-Tree is the default: Supports equality, range, sorting, and prefix LIKE. The workhorse index for analysts.

Leftmost prefix rule: Composite index (A, B) helps A-only queries but NOT B-only queries. Column order matters.

Partial indexes save space: Index only the rows you actually query. Perfect for 'active records' patterns.

Measure before and after: Always EXPLAIN ANALYZE before creating an index and after. Not all indexes help.

Indexes slow writes: Each index adds overhead to INSERT/UPDATE/DELETE. Only index what you need.

Check pg_indexes for existing indexes: Avoid creating redundant indexes. A composite index may already cover your query.

What is Next

Tomorrow we enter earlier: Analytics Power Tools. First up: Pivoting, Unpivoting, and the FILTER clause -- transforming rows into columns and back for the reports your stakeholders actually want to see.